STRATEGY PAPER | DECISION DOCUMENT

Consolidating a Fragmented DevOps Estate

A 12-month plan to migrate and consolidate 5 Azure DevOps organisations, ~1,000 repositories and ~1,500 pipelines onto a governed GitHub Enterprise platform, with a product-model internal developer platform and trustworthy engineering analytics.

Item	Detail
Audience	CTO, VP Engineering, Product leadership, Security/CISO, Finance, Enterprise Architecture
Status	Draft for decision. No migration or tooling change has been executed under this paper.
Decision requested	Approve the 90-day foundation and pilot, the measurement standard, and the non-waivable controls. Defer full-scale funding to the Day-90 gate.
Evidence basis	The supplied case-study brief (organisation, repository, pipeline counts; migration status; developer friction; disputed metrics). All other figures are labelled assumptions or proposed targets.
Date	24 October 2026

How to read this document
Given = stated in the case-study brief. **Assumption** = my working estimate, to be calibrated in the first 30 days. **Proposed target** = a goal, not a measured result. **Not verified** = a capability that requires enterprise configuration I could not test. Nothing in this paper should be quoted externally as measured performance.

The decision in one paragraph

Do not pause the transformation, and do not accelerate the whole programme. **Re-scope it into a gated migration factory**: fix measurement and controls first, prove the path on a small pilot, then move waves behind objective exit criteria. Repository migration is ahead of plan and should continue. Pipeline conversion is behind and should be replaced by a reusable golden path rather than converted repository-by-repository. Security enforcement should tighten now, because every additional unmanaged pipeline increases the cost of tightening later. Legacy-runner exceptions should be allowed but ring-fenced, owned and expiring.

1. Executive summary

Five separate Azure DevOps organisations, roughly a thousand repositories and fifteen hundred pipelines have produced a platform estate that is expensive to operate, difficult to secure consistently, and hard to measure. The organisations encode the same intent many times over: build, test, scan, deploy. Because each team owns its own copy, every improvement must be applied a thousand times, and every defect is discovered a thousand times.

The strategic problem is therefore not 'move to GitHub'. It is **consolidating duplicated engineering intent into a small number of governed, reusable paths**, and then making the resulting data trustworthy enough for leadership decisions. Tool migration is the vehicle; duplication removal, control consistency and measurement credibility are the actual outcomes.

What leadership is being asked to believe

Claim	Why it is credible	How we will prove it
Consolidation reduces cost and risk	Five organisations mean five sets of identity, policy, billing and audit boundaries; roughly 1,500 pipelines imply heavy copy-paste variation.	Duplicate-pipeline analysis in Wave 0; reduction in unique pipeline definitions and policy exceptions by Day 90.
A golden path beats mass conversion	Converting 1,500 pipelines one by one preserves duplication. A reusable template converts once and is inherited.	Share of pipelines on the golden path; time-to-first-green for a new service.
Metrics must be fixed before they are published	Metric quality is already disputed. Publishing a disputed number into a leadership dashboard destroys trust faster than publishing nothing.	Documented metric dictionary, join-coverage report, and a signed reconciliation record.
Security can be enforced without stopping delivery	Controls are codified once in the reusable workflow and rulesets, not negotiated per team.	Policy coverage percentage and count of active exceptions with expiry dates.

Recommended posture: re-scope, do not pause

The brief states that repository migration is ahead of plan while pipeline conversion is behind. Those two facts point in opposite directions, which is why the intuitive choices - pause everything, or push harder everywhere - are both wrong:

- **Continue** repository migration, because it is working and it is the prerequisite for consolidation.
- **Replace** pipeline-by-pipeline conversion with a reusable golden path plus a migration factory.
- **Tighten** security controls now on all newly migrated and newly created pipelines, with a dated backlog for the existing estate.
- **Pause only the wave** that depends on disputed metrics or on runners that cannot meet the control baseline - not the transformation.
- **Defer** org decommissioning until dependency, ownership and rollback evidence exists.

Twelve-month outcomes (proposed targets, to be calibrated in Days 1-30)

Outcome	Baseline	Proposed 12-month target	Measure of success
Consolidated source control	5 ADO organisations	1 GitHub Enterprise org, 0 remaining active ADO orgs	Org count, last-push date per org
Golden-path adoption	Assumed < 10%	>= 70% of active pipelines from reusable components	Pipeline inventory attribute
Security policy coverage	Assumed partial	100% of default branches protected; 100% new pipelines scanned	Ruleset + scan coverage report
Change failure rate	Disputed	Published only with quality flag; trend down	Metric dictionary + reconciliation log
Deployment frequency	Unknown	Established per-service baseline, then improve	Baseline table in Wave 0
Metric credibility	Disputed	One definition, one owner, one number per metric	Signed metric charter
Developer friction	Reported high	Measurable reduction in time-to-first-green	Platform telemetry + survey

The baseline column is deliberately empty of invented numbers. Filling it is the first deliverable of the programme, because every later benefit claim depends on it.

2. Context, evidence and working assumptions

What the brief tells us

- Five Azure DevOps organisations hold the current estate.
- Approximately 1,000 repositories and approximately 1,500 pipelines are in scope.
- DevOps practice maturity is low: gates, ownership and standards are inconsistent.
- Repository migration is ahead of plan; pipeline conversion is behind plan.
- Security is asking for stronger enforcement.
- Some legacy products cannot use standard hosted runners.
- Metric quality is disputed.
- Developers report added friction.

Working assumptions (to be validated in Wave 0, not presented as fact)

Assumption	Working value	Why it matters if wrong
Engineering population	400-600 engineers across 40-60 teams	Drives licence cost, training load and IDP adoption capacity.
Pipeline duplication	60-80% are near-copies of a small number of patterns	If duplication is low, the golden path saves less and waves shrink.
Pipeline age and ownership	A meaningful minority are unowned or dormant	Unowned pipelines must be quarantined, not migrated, to avoid carrying risk forward.
Legacy runner population	5-10% of pipelines need self-hosted or legacy runtime	Sets the size and cost of the exception fleet.
Azure Boards usage	Boards remains the work-tracking system of record for at least 12 months	Forces a coexistence and traceability design rather than a clean cutover.
Deployment definition	Many 'deployments' are actually build runs, not production releases	If unconfirmed, deployment frequency and change failure rate are not comparable across services.

Assumption	Working value	Why it matters if wrong
Metric dispute cause	Definition and join-key disagreement, not data volume	Determines whether the fix is governance or engineering.

Why the current estate is expensive

Cost driver	Mechanism	Leadership consequence
Duplication	Every pipeline re-implements build, test, scan and deploy logic	Improvements land slowly and inconsistently; defects multiply.
Five trust boundaries	Identity, permissions and audit are split five ways	No single security posture and no single evidence trail.
Weak gates	Quality and security checks are optional or inconsistent	Risk is discovered in production rather than in review.
Runner sprawl	Ad-hoc agents per team or per product	Patch drift, unknown inventory, and unclear ownership.
Unreliable metrics	Definitions and joins differ per team	Leadership cannot compare teams or track progress honestly.
Developer friction	Inconsistent templates and unclear ownership	Delivery slows and platform trust erodes.

Definition of done for the twelve months

The programme is complete when: one source-control organisation holds the active estate; new services are created from a golden path in under an hour; every default branch enforces the same reviewed check set; legacy runners exist only as named, owned, expiring exceptions; every published DORA metric has a definition, an owner, a quality flag and a reconciliation record; and the platform team can show its own service levels and adoption numbers.

3. Strategy principles and non-waivable controls

Principles are only useful if they settle arguments. Each of the following is written so that a team can predict the answer to a design question without escalating it.

#	Principle	What it settles in practice
1	Consolidate intent, not just repositories	One reusable definition per pipeline pattern; variation must be a parameter, not a fork.
2	Golden path by default, escape hatch by exception	Teams do not start from a blank file. Deviations require an owner and an expiry date.
3	Security is inherited, not negotiated	Scanning, provenance and least-privilege are baked into the template so teams cannot forget them.
4	Measure once, publish once	One metric definition, one owner, one number. A second definition is a defect.
5	Coexistence is designed, not improvised	Azure Boards, ADO pipelines and GitHub Actions have a documented interim contract and a retirement date.
6	Migrate in waves behind gates	No wave starts because a date arrived; it starts because entry criteria are met.
7	Platform is a product with customers	Adoption and satisfaction are success measures, not compliance counts.
8	No unowned asset survives	Repositories, pipelines and runners without an owner are quarantined, not migrated.
9	Automate judgement last	AI may diagnose, propose and summarise. It may not approve its own production change.

#	Principle	What it settles in practice
10	Evidence over assertion	If we cannot show the log, the report or the measurement, we do not claim it.

Non-waivable controls

> **These five controls cannot be traded for delivery speed**
> 1. Every default branch is protected by a reviewed ruleset with required checks. 2. No long-lived credentials in pipelines; workload identity federation for cloud access. 3. Secret scanning and push protection enabled; a detected secret is revoked, not just deleted. 4. Third-party build steps are pinned and dependency changes are reviewed before merge. 5. Production deployment requires an attributable approval trail. Exceptions may relax *which* check runs, never *whether* it is recorded.

Explicitly out of scope for year one

- Re-platforming application architectures. The programme changes how software is built and released, not what it is.
- Retiring Azure Boards. Work-item traceability is preserved; migration is a separate decision.
- A custom developer portal build from scratch. Buy or adopt before building.
- Autonomous production change by AI agents.
- Chargeback or showback as a funding mechanism. Cost visibility first, billing model later.

4. Target architecture

The target is one governed delivery platform with clear trust boundaries. The table below sets out the intended layers; each layer has an owner and a control set.

Layer	Target component	Primary control	Owner
Work tracking	Azure Boards (system of record), linked to GitHub via AB# references and status checks	Traceability from work item to deployed change	Product / PMO
Source control	GitHub Enterprise, single organisation, EMU or enterprise-managed identities	Rulesets, CODEOWNERS, signed and linear history	Platform Engineering
CI/CD	GitHub Actions: reusable workflows + composite actions; environment protection for prod	Pinned actions, least-privilege tokens, required checks	Platform Engineering
Compute	GitHub-hosted runners by default; governed self-hosted pools for legacy workloads	Ephemeral runners, patched images, network isolation	Platform Engineering + Security
Identity	OIDC federation to cloud providers; no stored long-lived secrets	Short-lived, scoped, auditable credentials	Security
Artifacts	Central package and container registry with provenance	Signed artifacts, SBOM, retention policy	Platform Engineering
Security	CodeQL SAST, dependency and secret scanning, dependency review	Policy-as-code; blocking on high severity	Security
Analytics	Ingestion to Bronze/Silver/Gold, then Power BI semantic layer	Metric dictionary, quality flags, lineage	Data & Analytics
Observability of the platform	Adoption, failure, queue time and exception telemetry for the platform itself	Platform SLOs	Platform Engineering

Delivery flow (target)

Stage	What happens	Automated evidence produced
1. Intent	Work item accepted in Azure Boards with owner and service tag	Work-item link and service identifier
2. Change	Pull request from a branch; CODEOWNERS review required	Review record, signed commit, diff scope
3. Validate	Reusable workflow: restore, build, unit test, coverage, SAST, SCA	Build log, coverage file, SARIF, dependency diff
4. Package	Artifact produced once and promoted, not rebuilt per environment	Immutable artifact digest and SBOM
5. Deploy	Environment-scoped deployment with approval and OIDC credentials	Approval record, deployment event, actor
6. Observe	Deployment and incident events emitted to analytics	Deployment event with environment and service
7. Learn	DORA and platform metrics refreshed with quality flags	Quality-flagged metric dataset

Architecture decisions that need leadership endorsement

Decision	Chosen position	Rejected alternative and why
Destination platform	One GitHub Enterprise organisation	Five parallel destinations perpetuate five trust boundaries.
Pipeline strategy	Reusable workflows with parameterised templates	Repository-by-repository conversion of 1,500 pipelines repeats the duplication problem at higher cost.
Runner strategy	Hosted first, governed self-hosted pools for named exceptions	Unbounded self-hosted estate cannot be patched or inventoried reliably.
Work tracking	Retain Azure Boards for 12 months with enforced linking	Simultaneous tool and process change multiplies disruption and obscures cause of failure.
Analytics	Local, reproducible pipeline feeding Power BI; warehouse only if volume demands	Buying a warehouse before metric definitions are agreed locks in the wrong grain.

5. Target operating model

The operating model separates **who builds the platform** from **who builds on it**, and makes the boundary explicit so that escalations have a destination.

Accountability by area

Area	Platform Eng.	App teams	Security	Ent. Arch.	Data	DX / Product
Platform roadmap and backlog	A	C	C	C	C	R
Reusable workflow ownership	A/R	C	C	I	I	C
Service delivery and quality	C	A/R	I	I	-	I
Security policy definition	C	I	A/R	C	I	-
Exception approval	R	R	A	C	-	I
Metric definitions	C	C	C	C	A/R	C
Developer experience research	C	C	-	-	I	A/R
Migration wave scheduling	A/R	C	C	C	I	C
Legacy runner inventory	A/R	C	C	I	-	I

A = accountable, R = responsible, C = consulted, I = informed. One accountable owner per row.

Service tiers and commitments

Tier	Applies to	Support	Ship cadence	Change lead time target
Gold (golden path)	New services and migrated tier-1 products	Named platform contact, 1 business day	Weekly platform release	Template change to available: <= 5 business days
Silver (standard)	Existing services on shared templates	Queue-based, 3 business days	Fortnightly	<= 10 business days
Bronze (constrained)	Legacy products on self-hosted runners	Best effort, documented workaround	Monthly, security-first	<= 20 business days
Exception	Anything outside policy	Owner named, expiry dated	Review at expiry	No commitment beyond the exception window

Decision rights

Decision	Decides	Must consult	Escalation path
Adopt a new reusable workflow pattern	Platform Engineering	Security, App team representatives	Head of DevOps
Grant a security exception	Security	Platform Engineering, product owner	CISO
Approve a migration wave gate	Head of DevOps + Enterprise Architecture	Security, Data	CTO
Change a published metric definition	Data & Analytics owner	Platform, Finance, Product	CTO
Accept residual risk for a legacy runner	Product owner + Security	Platform Engineering	CISO
Decommission an ADO organisation	CTO	All of the above	Board (if material)

Support, escalation and exception handling

- **Single entry point:** all platform requests, defects and exceptions enter one backlog with a visible owner.
- **Exception contract:** every exception names the control relaxed, the compensating control, the owner, the expiry date and the remediation plan. Undated exceptions are rejected automatically.
- **Escalation:** unresolved at tier -> wave lead -> Head of DevOps -> CTO. Security exceptions escalate to the CISO independently of delivery pressure.
- **Review cadence:** weekly platform operations review, monthly steering group, quarterly executive checkpoint.
- **Handover:** no wave closes until runbooks, on-call ownership and dashboards are accepted by the receiving team.

6. Internal developer platform product model

Treat the platform as a product with named customers, a roadmap, adoption metrics and a service catalogue. This is what converts 'we built templates' into 'teams choose to use them'.

Personas and jobs to be done

Persona	Job to be done	Current friction (assumed)	Platform response
Application developer	Ship a change safely without becoming a pipeline expert	Copy an existing pipeline and hope	Golden path, scaffolding, local parity
Tech lead	Know the service is compliant and observable	Unclear what 'compliant' means	Visible checks, ownership metadata, scorecard
SRE / operations	Deploy and roll back predictably	Inconsistent environments and approvals	Environment protection, immutable artifacts
Security engineer	Prove controls across the estate	Manual sampling, no inventory	Policy-as-code, coverage reporting
Data / analytics	Trust the numbers leadership sees	Definitions differ per team	Metric dictionary, lineage, quality flags
Product manager	Understand delivery health	Disputed dashboards	One definition, one owner, one number

Service catalogue v1

Catalogue item	What the team gets	Time to value (target)
New service scaffold	Repository, baseline pipeline, tests, security defaults, ownership file	< 1 hour
Standard web service pipeline	Reusable build/test/scan/package workflow with parameters	< 30 minutes to first green
Deployment bundle	Environment definitions, approval gates, OIDC wiring	Same day
Legacy runner onboarding	Governed pool, patched image, inventory entry, exception record	5 business days
Metric onboarding	Service registered for deployment/incident events	1 sprint
Exception request	Structured request, compensating control, expiry	2 business days

Product lines, prioritisation and feedback

- **Product lines:** Delivery (CI/CD), Environments, Security enablement, Analytics, Self-service portal.
- **Prioritisation:** weighted shortest job first, scored on user reach, friction removed, risk reduced and effort. Security debt that blocks a non-waivable control pre-empts the queue.
- **Adoption funnel:** aware -> tried -> first green -> habitual -> advocate. Each stage has a metric; a large 'tried' with small 'habitual' indicates a product defect, not a training problem.
- **Feedback loops:** fortnightly developer council, embedded research during waves, platform NPS, automatic friction telemetry (failure reasons, retry counts, queue time).
- **Definition of a good platform change:** reduces steps for teams, reduces variance in outcomes, or increases evidence for auditors - ideally all three.

IDP measurements

Measure	Definition	Why leadership should care
Adoption rate	Active services on a golden path / active services	Real consolidation progress
Time to first green	Median elapsed time from scaffold to first successful pipeline	Direct friction reduction
Platform change lead time	Idea to available template change	How fast improvement reaches teams
Exception count and age	Open exceptions, median age, expired count	Control erosion indicator
Platform availability	Successful workflow runs / attempted runs	Reliability of the factory
Support load	Requests per 100 services	Scaling efficiency of the platform team

7. Migration and consolidation plan

Migration is organised as a factory with repeatable waves. Each wave has entry criteria, an exit gate and a rollback position. The factory, not the calendar, sets the pace.

Wave model

Wave	Scope	Entry criteria	Exit gate	Indicative timing
0 - Discovery	Inventory all 5 orgs; classify repos, pipelines, runners, owners, dependencies	Access granted; ownership contacts named	Signed inventory; duplication analysis; baseline metrics defined	Weeks 1-4
1 - Pilot	2-3 engaged teams, 1 non-critical product each	Golden path v1 available; controls agreed	Pilot services live; friction log; security baseline met	Weeks 5-10
2 - Reusable pipeline modernisation	Top 5-10 pipeline patterns cover the majority of the estate	Pilot lessons incorporated	Templates adopted by pilot; pattern coverage validated	Weeks 8-16
3 - Broad migration	Standard workloads in batches of 20-40 repos	Self-service migration playbook; capacity confirmed	Batch exit: build parity, tests, scans, rollback rehearsed	Months 4-8
4 - Complex and regulated	Tier-1 products, regulated data, unusual runtimes	Dedicated design per product; risk accepted	Product-specific acceptance; audit evidence complete	Months 6-10
5 - Legacy runner exceptions	Workloads that cannot use hosted runners	Documented technical constraint	Governed pool live; owner and expiry recorded	Months 5-11
6 - Decommission	Retire ADO orgs, agents and duplicate pipelines	No active dependency; retention satisfied	Org archived/closed; cost removed; evidence retained	Months 10-12

Selection criteria for the first waves

- **Choose the pilot by engagement, not by importance.** A willing team with a simple pipeline proves the path; a reluctant team with a complex pipeline proves nothing.
- **Exclude** services with unreviewed regulatory obligations, undocumented dependencies or no named owner.
- **Prefer** services whose pipeline is already close to the target pattern, so the first wave validates the template rather than the exception process.
- **Quarantine** dormant repositories with no commits in the agreed window instead of migrating them: archive first, migrate only on demand.

Coexistence and rollback

Concern	Interim position	Exit condition
Azure Boards	Remains the work-tracking system of record; GitHub PRs require AB# links	Product decision at month 9 based on usage data
ADO pipelines	Run in parallel with GitHub Actions per migrated service until parity is proven	Two clean releases or agreed stability window
Rollback	Keep ADO pipeline operational for a fixed period; migration reversible per service	Rollback window expires at wave exit
Identity	Dual identity tolerance during transition, with a hard end date	All access via enterprise identity
Reporting	Analytics ingests both sources with a source column	Single source after decommission

Legacy runner exceptions

Some products cannot use standard hosted runners. The answer is not to exempt them from control; it is to change the *shape* of the control:

- One governed pool per genuine constraint, not per team.
- Ephemeral or rebuilt-on-use images; no long-lived agent state.
- Network egress restricted and recorded; credentials via identity federation only.
- Named product owner, documented technical justification, expiry date, quarterly re-justification.
- Security review at onboarding and at every renewal. An expired exception stops deployment automatically.

Decommissioning criteria

- No successful pipeline run in the retention window, or every run replaced by a GitHub equivalent.
- No repository push in the defined dormancy window, or contents archived with a restoration test.
- Audit evidence exported and retained to policy.
- Billing and licence impact confirmed by Finance.
- Named executive sign-off, because decommissioning is the only irreversible step in the programme.

8. Governance, security and trusted metrics

Control catalogue

Control	Classification	Implementation	Evidence
Default-branch protection	Mandatory	Rulesets: PR required, reviewed check set, no force push, no deletion	Ruleset configuration export
Required status checks	Mandatory	Build, test, coverage, SAST, SCA as applicable	Check run history
Least-privilege workflow token	Mandatory	Workflow-level read default; elevated per job only	Workflow definition review
No long-lived cloud secrets	Mandatory	OIDC federation to cloud providers	Federation configuration
Secret scanning and push protection	Mandatory	Repository and organisation settings	Scanning alerts and response log
SAST	Mandatory	CodeQL default setup or approved equivalent	Analysis results
Dependency vulnerability review	Mandatory	Audit in build plus PR dependency review	Scan output

Control	Classification	Implementation	Evidence
Action pinning	Mandatory (new), planned (existing)	Full commit SHA for third-party actions	Pinned reference scan
Artifact provenance and SBOM	Recommended	Registry provenance; SBOM per release	Attestation record
Environment approvals for prod	Mandatory	Protected environments with reviewers	Approval audit trail
Ownership metadata	Mandatory	CODEOWNERS and a service ownership file	Coverage report
Runner inventory and patch level	Mandatory (self-hosted)	Central inventory with image version	Inventory report

Metric definitions and anti-gaming

Metric	Definition	Denominator / scope	Quality stance
Deployment frequency	Successful deployments to production per service per period	Production environments only, service-level	Unavailable until deployment events are distinguishable from build runs
Lead time for changes	Median time from commit merged to production deployment	Merged commits that reached production	Unavailable where no reliable commit-to-deployment join exists
Change failure rate	Deployments causing an incident of agreed severity / deployments	Same services, same window	Published as a proxy until incident-to-deployment linking is event-level
Failed deployment recovery time	Median time from failure detection to service restoration	Failed deployments only	Requires incident records with start and resolution
Platform metrics	Adoption, time to first green, exception age, check pass rate	Platform scope	Measured from platform telemetry

*Where the source data cannot support a metric, the metric is published as **unavailable** or **limited**. An honest gap is recoverable; a confidently wrong number is not.*

Reconciling disputed metrics

- **Freeze publication** of contested metrics until one definition is signed by Data, Platform and Product.
- **Trace end to end**: source event -> raw layer -> transformation -> metric -> dashboard. Every step gets an owner and a test.
- **Publish the exclusion list** alongside the number: which services, environments and periods are missing and why.
- **Reconcile in public**: a written comparison of the old and new numbers, with the cause of difference.
- **Attach a quality flag** to every value (validated, limited, unavailable) and make it visible in the dashboard.
- **Review quarterly** with a named decision owner; changes are versioned, never silently edited.

Preventing metric gaming

- Never rank individuals on DORA metrics; rank only service-level trends with context.
- Pair speed metrics with stability metrics so improving one at the expense of the other is visible.
- Publish denominators and exclusions next to every figure.
- Detect anomalies in volume as well as in rate: a sudden drop in deployments is a signal, not a success.
- Rotate the metric review panel so no single team owns the narrative about its own performance.

9. AI Ops and agentic DevOps

AI is useful here for one specific reason: this estate generates enormous repetitive signal - failed pipelines, duplicated patterns, review comments, runbooks. The opportunity is to compress the human time spent interpreting that signal, not to remove human accountability.

Use case	Value	Risk tier	Guardrails required
CI failure diagnosis	Explains failures and proposes a fix from logs and recent diffs	Low	Read-only logs; suggestion only; no automatic push
Runbook and policy retrieval	Answers operational questions from approved internal sources	Low	Curated corpus; citations required; no external data egress
Test generation	Drafts unit tests for changed code	Medium	Generated tests reviewed by a human before merge
Pull-request review assistance	Flags risky diffs and missing tests	Medium	Advisory only; cannot approve its own suggestion
Infrastructure and pipeline review	Detects policy drift in workflow definitions	Medium	Read-only analysis; proposals via PR
Migration assistance	Converts a pipeline to the golden path for human review	Medium	Output reviewed; never merged unreviewed
Remediation execution	Applies approved fixes to non-production	High	Scoped identity, approval gate, audit trail, blast-radius limit
Production change	Any autonomous action in production	Prohibited in year one	Not enabled

Operating rules for AI in the platform

- **Least privilege by default:** agents get read-only access unless a specific, reviewed use case requires otherwise.
- **Separate identity:** every agent has its own identity, credentials and audit trail; it never borrows a human's.
- **Human approval for production:** no agent-initiated production change without an attributable human approval.
- **Data boundaries:** only approved repositories and sources; no customer data, secrets or personal data in prompts.
- **Evaluation before rollout:** each use case is scored on accuracy, false-positive rate, cost and time saved against a labelled sample.
- **Auditability:** prompts, tools called and outputs retained for review; a rollback path exists for every automated action.

Why start narrow
The fastest way to lose trust in AI Ops is a confident wrong diagnosis in a production incident. Start with read-only diagnosis and retrieval, publish measured accuracy, and only widen permissions when the evidence supports it.

10. Twelve-month roadmap

Quarter	Theme	Key milestones	Dependencies	Gate
Q1 (M1-3)	Foundation and pilot	Inventory complete; metric charter signed; golden path v1; 2-3 pilot services live; control baseline defined; exception policy approved	Access to all orgs; named product owners; security sign-off	G1: pilot proves path and controls
Q2 (M4-6)	Scale the factory	Self-service migration playbook; reusable templates for top patterns; 25-40% of standard workloads migrated; Azure Boards linking enforced	Pilot retrospective; runner capacity; training	G2: throughput and quality stable
Q3 (M7-9)	Broad migration and analytics	60-75% migrated; governed legacy runner pools; DORA dashboard with quality flags published; AI Ops read-only use cases in production	Metric reconciliation; runner exceptions approved	G3: analytics trusted, migration on track
Q4 (M10-12)	Consolidate and decommission	90%+ migrated; ADO orgs decommissioned; legacy agents retired; platform SLOs operating; year-2 plan	No active dependencies; audit evidence retained; Finance sign-off	G4: consolidation complete

Staffing assumptions (to be confirmed against actual capacity)

Role	FTE (suggested)	Focus
Platform engineering lead	1	Architecture, reusable workflows, runner strategy
Platform engineers	3-4	Templates, migration factory, self-service
Migration engineers (virtual team)	4-6	Wave execution, drawn from product teams
Security engineer (embedded)	1	Policy-as-code, exception review, scanning
Data / analytics engineer	1-2	Pipeline, metric dictionary, Power BI model
Developer experience / product	1	Personas, research, adoption, roadmap
Programme manager	1	Waves, dependencies, reporting, risk

The virtual migration team is the lever: seconding product engineers for one wave builds advocacy and reduces the 'platform team does not understand our product' objection.

11. Prioritised 90-day plan

The first ninety days exist to answer one question: **can we prove the path and the controls on a small number of real services before committing the full estate?**

Window	Focus	Key deliverables	Exit criteria	Accountable
Days 1-14	Mobilise and discover	Access to 5 orgs; inventory extract; duplication analysis; ownership map; metric charter draft; risk register	Signed inventory; contested metrics identified	Head of DevOps
Days 15-30	Baseline and design	DORA baseline (with quality flags); target architecture; control catalogue; exception policy; golden path design	Baseline published internally; design reviewed	Platform lead
Days 31-60	Build and pilot	Golden path v1; reusable workflow; rulesets; secret scanning; SAST/SCA; 2-3 pilot services migrated	Pilot builds, tests and deploys with all controls	Platform lead + pilot owners
Days 61-75	Validate and measure	Friction log; time-to-first-green; security coverage report; rollback rehearsal; developer feedback	Pilot meets agreed thresholds	DX lead
Days 76-90	Decide and scale	Wave 2 plan; capacity model; funding case; AI Ops read-only pilot proposal; leadership review	Gate decision taken with evidence	Head of DevOps + CTO

Day-90 gate criteria

Criterion	Threshold (proposed)	If not met
Pilot services live on the golden path	All pilot services build, test, scan and deploy	Do not scale; fix the path
Controls exercised end to end	Branch ruleset, SAST, SCA, secret scanning, OIDC deploy observed	Escalate to Security; block wave 2
Time to first green	Under 1 hour for a new standard service	Treat as product defect, not training issue
Rollback proven	At least one rollback rehearsed and timed	Rehearse before broad migration
Metric baseline agreed	Signed definitions and quality flags for all reported metrics	Publish nothing contested
Developer feedback	Pilot teams recommend adoption	Redesign before scaling

12. Priorities, investment and what to stop

Prioritise

Item	Why now	Opportunity cost of delay
Measurement foundation	Every benefit claim and every funding decision depends on it	Continued disputed reporting; decisions made on noise
Golden path and reusable workflows	Converts 1,500 individual problems into a handful of shared ones	Permanent duplication; each future change costs 1,500 times
Security baseline on new work	Cheapest moment to enforce is at creation	Retrofitting controls across a growing estate
Pilot with willing teams	Produces evidence and advocates cheaply	Scaling an unproven path and losing credibility
Runner inventory and governance	Unknown runners are unknown risk	Unpatchable estate; audit failure

Stop

Stop	Rationale	Minimum safe replacement
Pipeline-by-pipeline conversion	Preserves duplication at high cost and cannot keep pace	Reusable template plus guided self-service
Creating new pipelines outside the golden path	Every exception adds long-term cost	Documented exception with owner and expiry
Per-team runner sprawl	Cannot be patched, inventoried or audited	Governed pools per genuine constraint
Publishing contested metrics	Destroys trust faster than publishing nothing	Quality-flagged metrics with exclusions
Unbounded security exceptions	Accumulated risk with no owner	Time-boxed, owned, compensating-controlled exceptions

Defer

Deferred	Until	Why deferring is safe now
Retiring Azure Boards	Month 9 decision	Simultaneous tool and process change obscures cause; traceability is preserved meanwhile
Full org decommissioning	Months 10-12	Irreversible; requires dependency and audit evidence
Custom developer portal build	After catalogue proves demand	Buy or adopt first; building early locks in assumptions
Autonomous production change	Year 2 at the earliest	Accountability and blast-radius controls are not mature
Chargeback	Year 2	Cost visibility first; billing models need stable ownership data
Multi-cloud deployment standardisation	Year 2	Not the current constraint; would add scope without removing duplication

Indicative investment shape

No monetary figures are asserted here, because I do not have verified licence, run-rate or headcount data. The investment is best framed as three buckets to be costed in Days 15-30: **(1) platform capability** - the standing team; **(2) migration throughput** - seconded engineers and wave effort; **(3) tooling and infrastructure** - licences, hosted minutes, self-hosted capacity, analytics. The funding decision at Day 90 should be based on measured pilot throughput extrapolated across the inventory, not on a top-down estimate.

13. Risks and mitigations

Risk	Likelihood	Impact	Mitigation	Owner
Migration stalls behind pipeline conversion	High	High	Replace mass conversion with golden path and factory; track throughput not dates	Head of DevOps
Security controls slow delivery and drive shadow pipelines	Medium	High	Inherit controls in templates; measure friction; keep exceptions possible but owned	CISO + Platform
Disputed metrics undermine the programme	High	High	Metric charter before publication; quality flags; public reconciliation	Data lead
Legacy runner estate grows unchecked	Medium	High	Inventory, expiry-dated exceptions, quarterly re-justification	Platform + Security
Unowned repositories and pipelines migrate anyway	Medium	Medium	Quarantine policy; ownership required for migration	Platform lead
Platform team becomes a bottleneck	High	Medium	Self-service, virtual migration team, tiered support	Head of DevOps

Risk	Likelihood	Impact	Mitigation	Owner
Key-person dependency on a few engineers	Medium	Medium	Pairing, documented runbooks, rotation	Engineering managers
Cost overrun on licences or runner minutes	Medium	Medium	Inventory-driven capacity model; usage dashboards; gate at Day 90	Finance + Platform
Developer trust erosion	Medium	High	Developer council, published friction metrics, visible fixes	DX lead
AI Ops incident damages confidence	Low	High	Read-only start, evaluation harness, human approval, audit trail	Platform + Security

14. Decisions requested

#	Decision	Owner	Needed by	Consequence of delay
1	Adopt re-scoped, gated migration rather than pause or full acceleration	CTO	Day 5	Waves continue without evidence; funding follows opinion
2	Sign the metric charter and freeze contested metrics until reconciled	CTO + Data + Product	Day 20	Leadership continues to see disputed numbers
3	Approve the five non-waivable controls, including time-boxed exceptions	CISO + CTO	Day 15	Security posture remains inconsistent as estate grows
4	Fund the 90-day foundation and pilot (team, access, tooling)	CTO + Finance	Day 10	No pilot evidence exists for the Day-90 gate
5	Endorse the legacy runner exception policy and ownership model	CISO + Product	Day 30	Unbounded runner sprawl becomes permanent
6	Confirm Azure Boards as system of record for 12 months	Product leadership	Day 30	Traceability design churns; migration scope reopens

What would make me change this recommendation

- If inventory shows very low pipeline duplication, the factory model loses its main advantage and waves should be smaller.
- If the pilot cannot meet the control baseline without unacceptable friction, the golden path design is wrong and must be redesigned before scaling.
- If metric reconciliation shows the source data cannot support reliable DORA measures at all, publishing them should be abandoned for the year rather than fudged.
- If security cannot support time-boxed exceptions, migration throughput will fall sharply and the wave plan must be re-sequenced, not the controls weakened.

15. Assumptions, limitations and confidence

This paper contains no screenshots, invented baselines or claimed deployments. Targets are proposals; constraints come from the brief. The Day-90 gate exists because today's evidence cannot justify committing the full estate.

Area	Confidence	Basis
Organisation, repository and pipeline counts	Stated	Supplied case-study brief
Pipeline duplication estimate	Low	Not measured; requires Wave 0 inventory
Engineering population and team count	Low	Assumed to size effort only
Security control feasibility	Medium	Standard platform capabilities; enterprise settings not verified
Metric availability	Low	Depends on event-level joins that may not exist
Roadmap timing	Medium	Indicative; gated by evidence rather than calendar
Cost figures	Not provided	No verified licence or run-rate data available